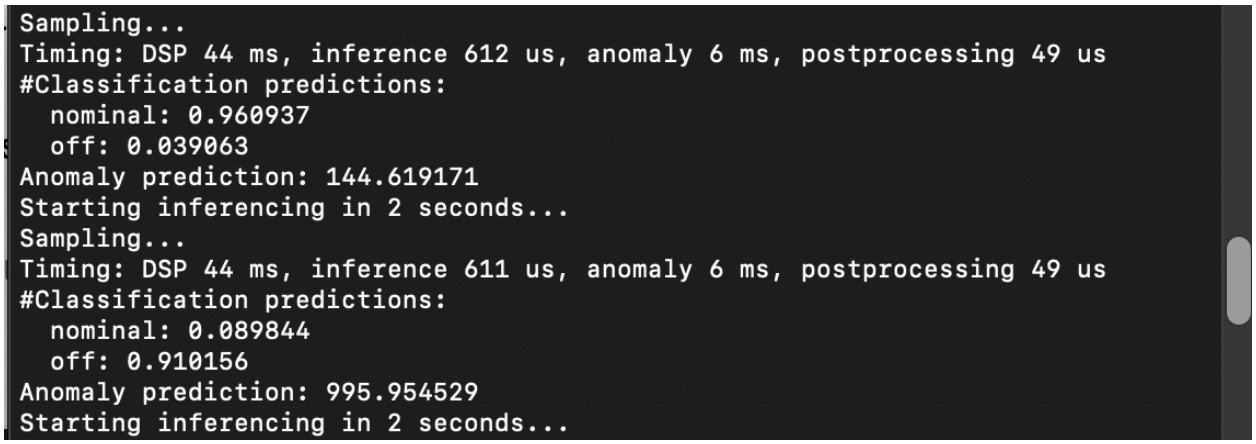
Part 1
Edge Impulse Link: https://studio.edgeimpulse.com/public/989446/live

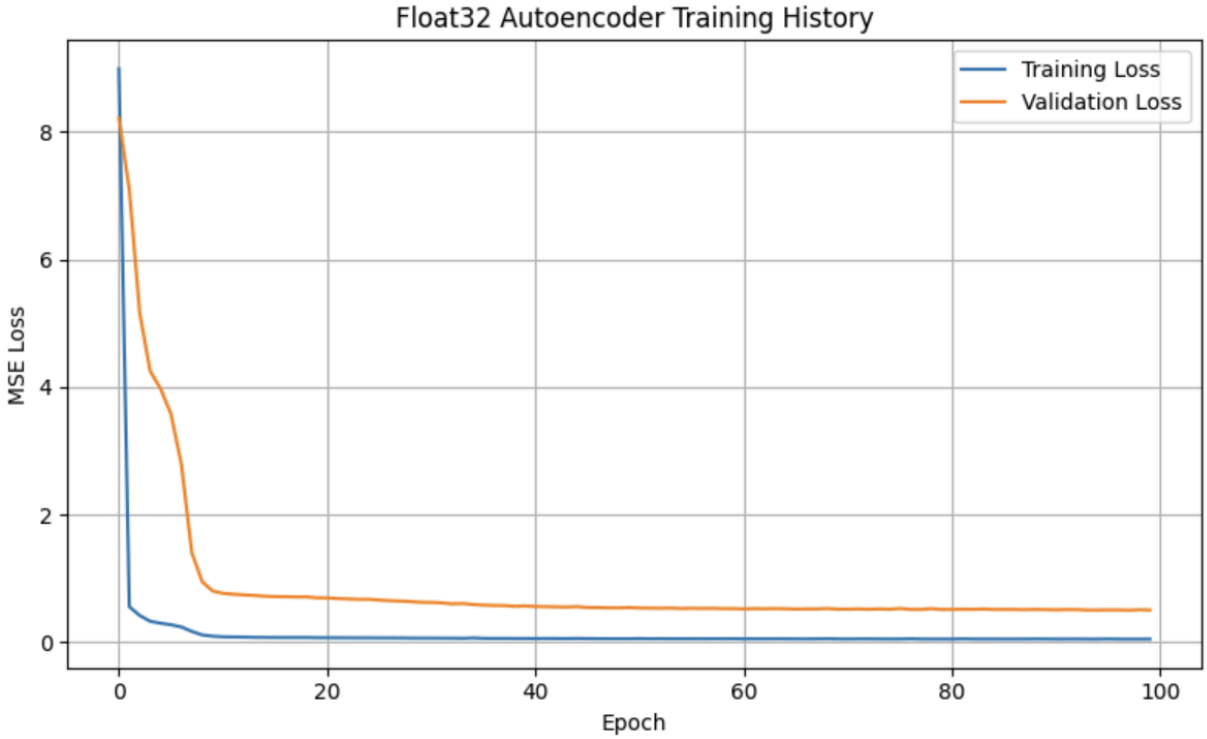
1. accX, accY, accZ
2. The features are a mix of time domain and frequency domain with a total of 33 features, such as accX RMS, accX Peak 1 Height, accY Peak 3 Height, accX Peak 1 Freq, accY RMS
3. The output is nominal and off.
4. The accX RMS is a time domain feature, the rest are frequency domain features. There are 4 features used: accX RMS, accX Peak 1 Height, accY Spectral Power 2.0 - 5.0 Hz, accZ Spectral Power 2.0 - 5.0 Hz
5. screenshot:

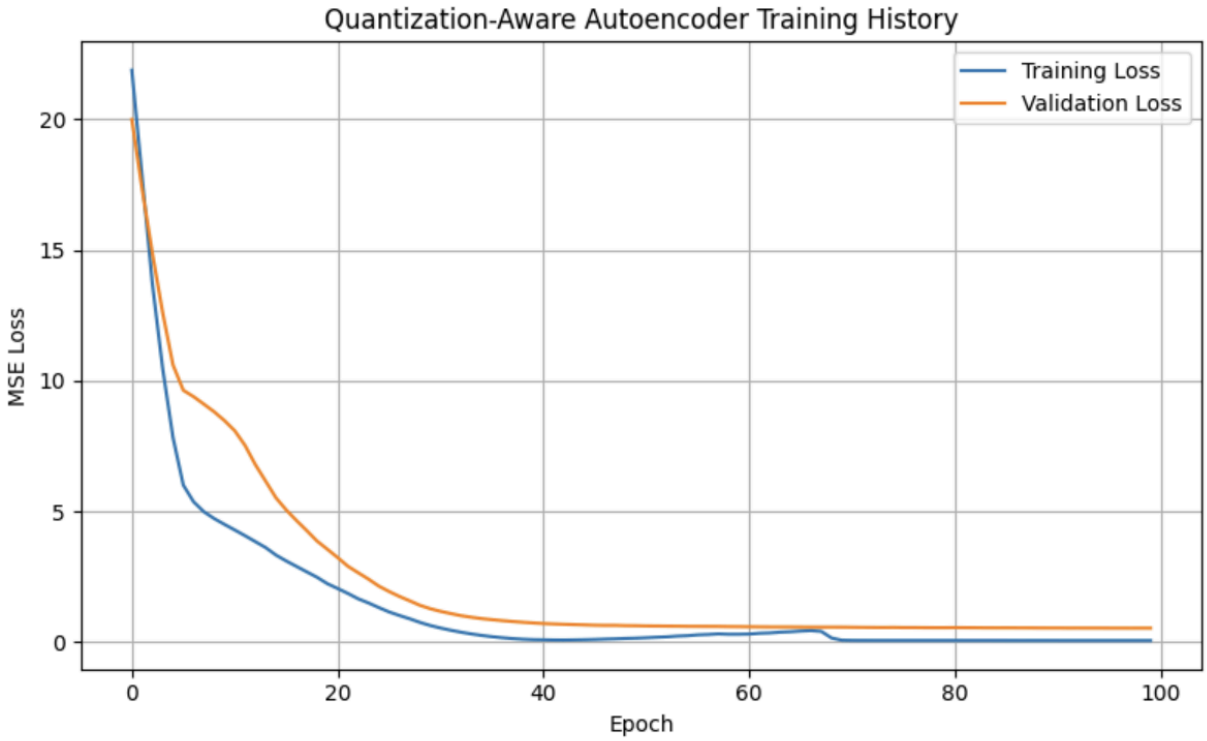


6. The first case shows that it is 96.0937% confident that the device is nominal, which is when the device is steady. Then the second case shows that the device is 91.0156% off, meaning it is probably moving. The output should not always be trusted, because there is an anomaly prediction score that shows the results might be unusual.

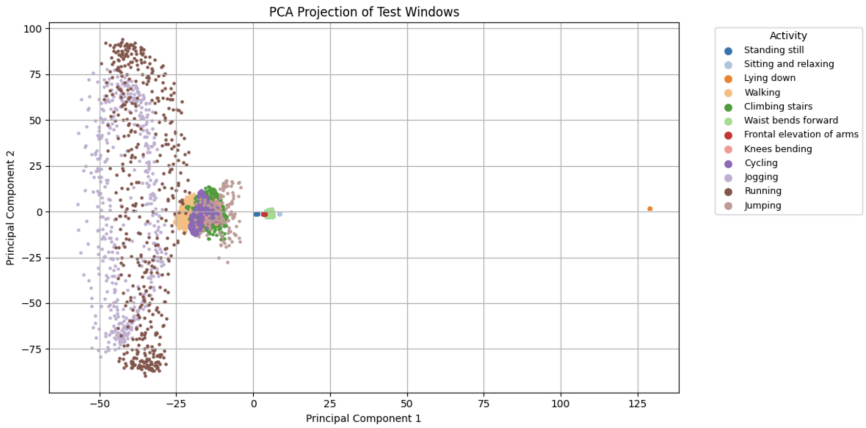
Part 2

Training and validation loss curves

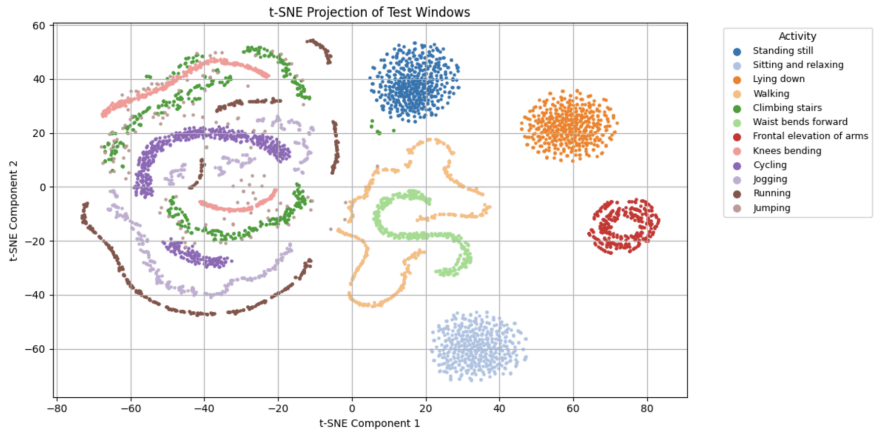




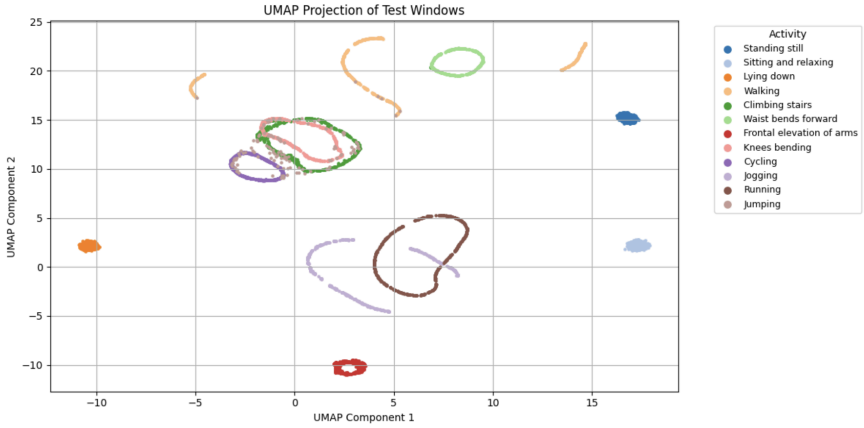
PCA visualization



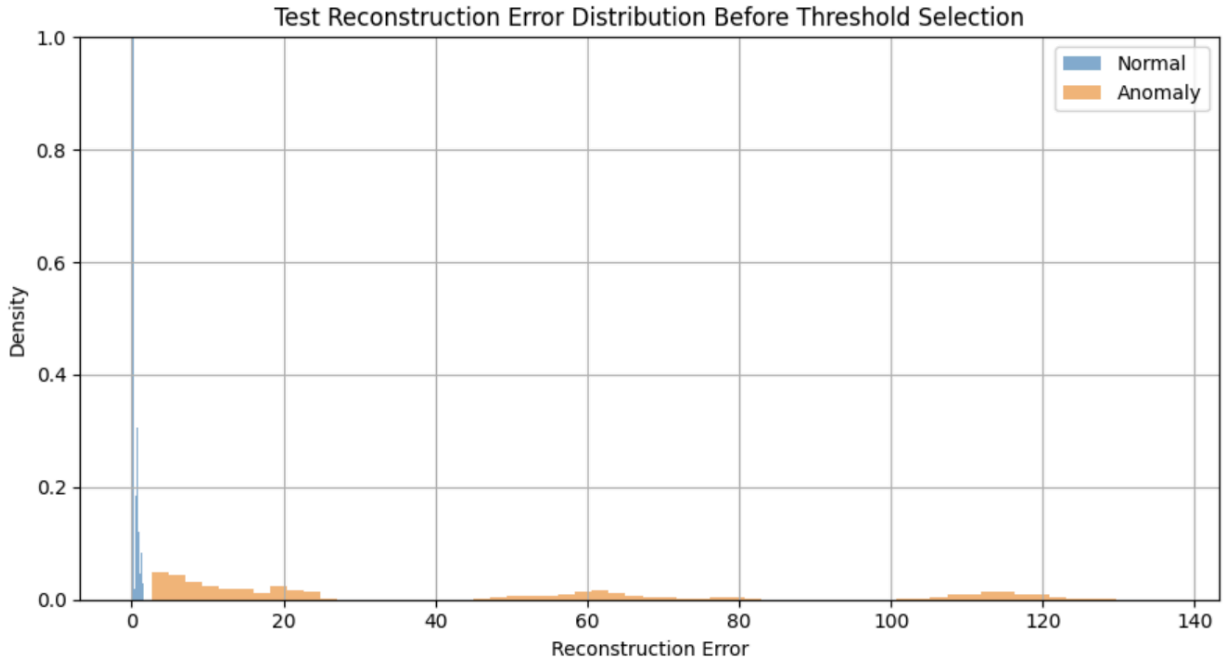
t-SNE visualization

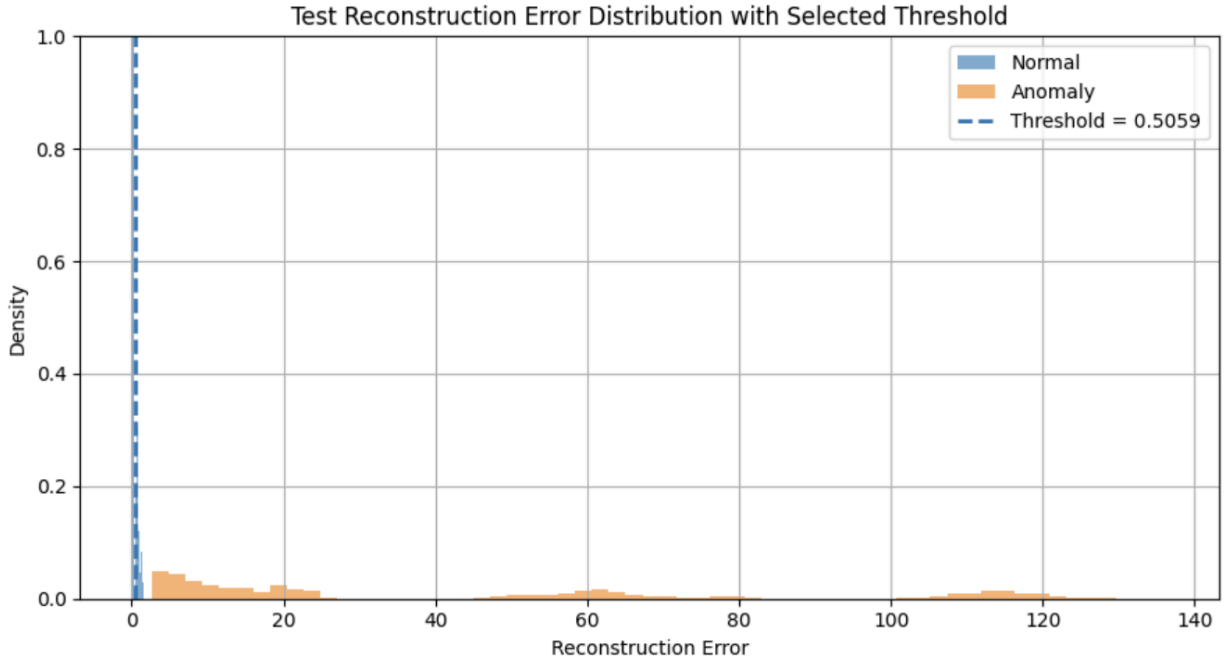


UMAP visualization

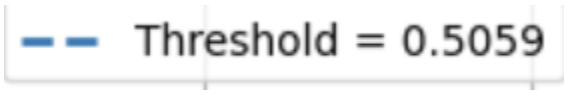


Reconstruction error distribution

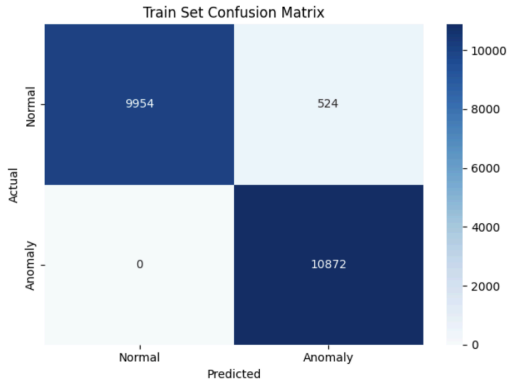


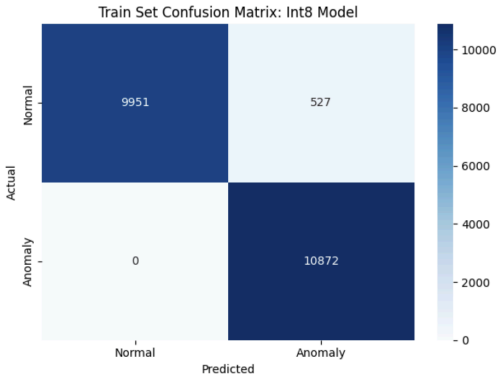


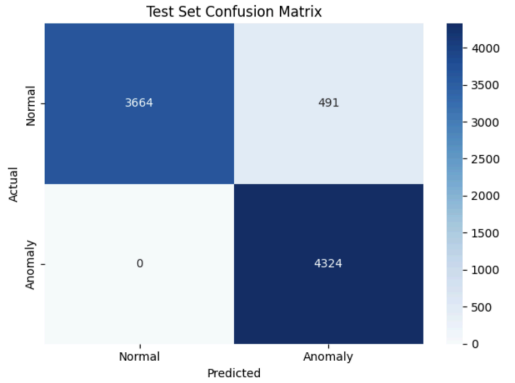
Selected anomaly threshold

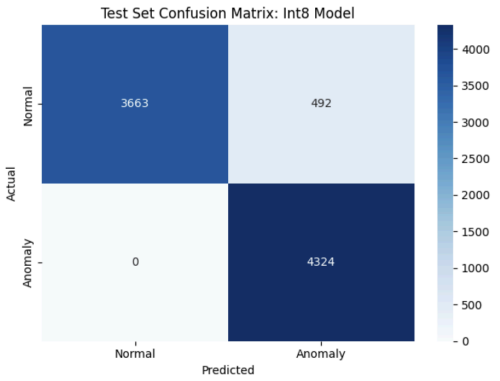


Confusion matrix









Classification report or evaluation metrics

Train set evaluation

	precision	recall	f1-score	support
Normal (0)	1.00	0.95	0.97	10478
Anomaly (1)	0.95	1.00	0.98	10872
accuracy			0.98	21350
macro avg	0.98	0.97	0.98	21350
weighted avg	0.98	0.98	0.98	21350

Test set evaluation

	precision	recall	f1-score	support
Normal (0)	1.00	0.88	0.94	4155
Anomaly (1)	0.90	1.00	0.95	4324
accuracy			0.94	8479
macro avg	0.95	0.94	0.94	8479
weighted avg	0.95	0.94	0.94	8479

Train set evaluation: int8 model

	precision	recall	f1-score	support
Normal (0)	1.00	0.95	0.97	10478
Anomaly (1)	0.95	1.00	0.98	10872
accuracy			0.98	21350
macro avg	0.98	0.97	0.98	21350
weighted avg	0.98	0.98	0.98	21350

Test set evaluation: int8 model

	precision	recall	f1-score	support
Normal (0)	1.00	0.88	0.94	4155
Anomaly (1)	0.90	1.00	0.95	4324
accuracy			0.94	8479
macro avg	0.95	0.94	0.94	8479
weighted avg	0.95	0.94	0.94	8479

Arduino serial monitor output showing reconstruction error and normal/anomaly detection results

Normal:

```
13:37:43.409 -> Result: Normal activity
13:37:43.441 -> Reconstruction error: 0.101910
13:37:43.441 -> Result: Normal activity
13:37:43.474 -> Reconstruction error: 0.101907
13:37:43.474 -> Result: Normal activity
13:37:43.507 -> Reconstruction error: 0.101908
13:37:43.507 -> Result: Normal activity
13:37:43.540 -> Reconstruction error: 0.101906
13:37:43.540 -> Result: Normal activity
13:37:43.573 -> Reconstruction error: 0.101907
13:37:43.573 -> Result: Normal activity
```

Anomaly:

```
13:39:05.685 -> Reconstruction error: 0.667886
13:39:05.685 -> Result: Anomaly detected
13:39:05.718 -> Reconstruction error: 0.669500
13:39:05.718 -> Result: Anomaly detected
13:39:05.750 -> Reconstruction error: 0.649980
13:39:05.750 -> Result: Anomaly detected
13:39:05.750 -> Reconstruction error: 0.649568
13:39:05.782 -> Result: Anomaly detected
13:39:05.782 -> Reconstruction error: 0.636098
13:39:05.782 -> Result: Anomaly detected
13:39:05.814 -> Reconstruction error: 0.655463
13:39:05.814 -> Result: Anomaly detected
```

Part 3

• What threshold did you choose for anomaly detection, and why?

I chose the threshold to be 0.5059, because I set that 95% of normal training data are below this threshold and only the top 5% are considered anomalies. So it lies just above the normal distribution and separates the anomalies.

• How does reconstruction error help distinguish normal and anomalous activity?

Normal activities produce low reconstruction error because their patterns are familiar to the model, while anomalous activities produce higher reconstruction error because their motion patterns are different from the normal training data. So the windows with reconstruction error above the selected threshold are classified as anomalies.

• What information from the Python notebook must be preserved when deploying the model to Arduino?

The trained int8 TFLite model, input window size, feature ordering, preprocessing steps, quantization parameters, and reconstruction-error threshold must be preserved.

• Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

Because the autoencoder was trained on a specific input format and data distribution. The model expects each input to be a 100-sample accelerometer window flattened into a 300-element vector used during training. If the Arduino uses different preprocessing assumptions, normal activity can be falsely detected as anomalies.

• What are the main limitations of this anomaly detection method when deployed on a microcontroller?

The main limitation is that its performance depends on matching the deployment conditions to the notebook training conditions. The Arduino must use the same preprocessing assumptions. If the sensor placement or motion pattern changes, the reconstruction-error distribution may change and the selected threshold may no longer be able to classify the normal and anomaly activities.